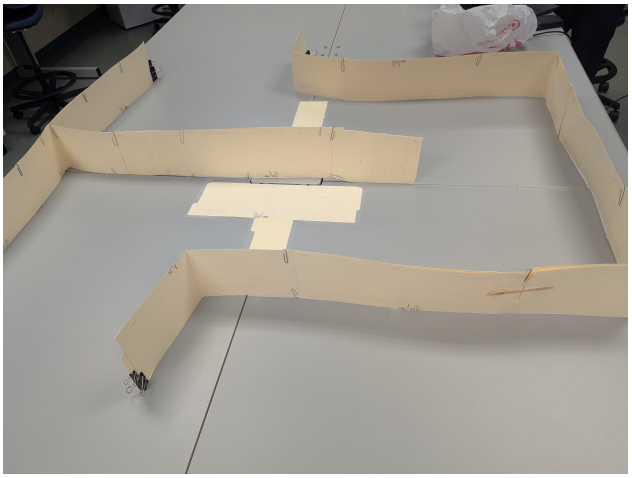
Microcontrollers Project Report
EEGR3233

Cooper McAllister

April 30 2026

1 Introduction

This was an individual project; therefore, all the work was done by Cooper McAllister.
The objective of this project was to create a robot that could autonomously drive through a simple maze without colliding with the walls. Students were given a robot base with two servo motors, a number of distance sensors, and a microcontroller. The maze is shown below:

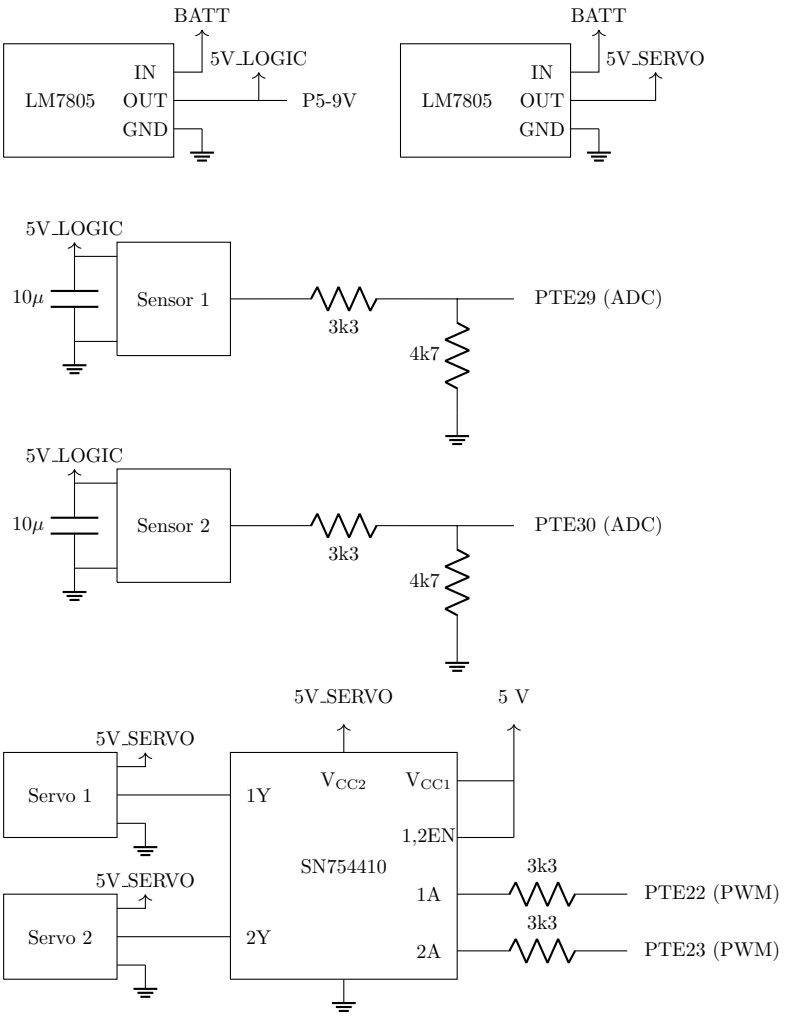


2 Hardware Design

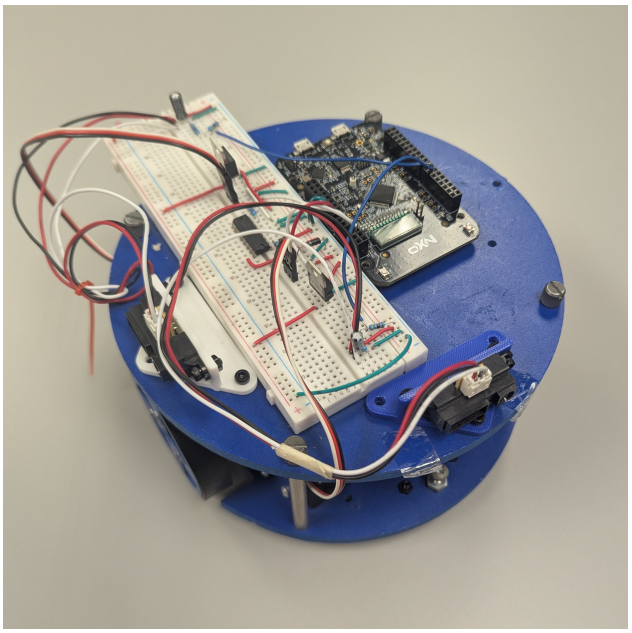
The main goals of the hardware design for this project were ensuring reliability and protecting the microcontroller (MCU) from current or voltage conditions outside of its rated values. There are two separate LM7805 voltage regulators: one for the high-power servos, and one for the low-power distance sensors and MCU. The sensors are connected to the MCU by means of a resistor network that ensures that the voltage entering the MCU pins cannot exceed their rated voltage of 3.3V, even if the sensors output a voltage spike of 5V. This can be proved using voltage division:

$$V_{port,max} = 5\text{V} \cdot \frac{4.7\text{k}\Omega}{4.7\text{k}\Omega + 3.3\text{k}\Omega} = 2.9375\text{V} < 3.3\text{V}$$

Decoupling capacitors of 10μF are included for each sensor.
The servo motors are driven via an SN754410 driver. Although not strictly required for servo motors, this chip helps protect the MCU and ensure that enough current is provided to the servo motors. This chip is driven via resistors to limit the current flowing out of the MCU pins. Power is provided to the MCU via the P5-9V pin. The battery is 4 1.5V AA batteries in series, providing a total voltage of 6V.
The circuit schematic is shown below:

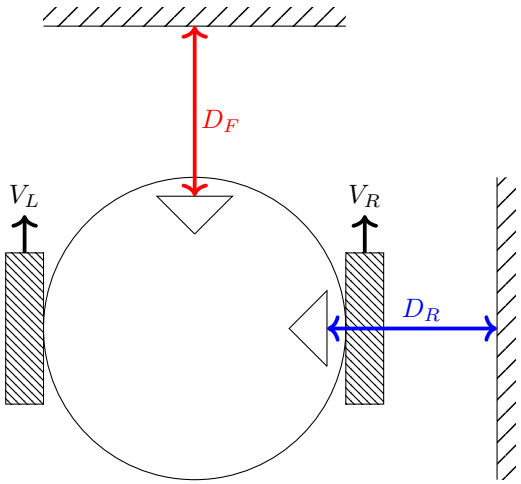


In addition to the electronics, which were implemented on a breadboard, mounts were designed and 3D printed to hold the distance sensors. All the parts of the robot, including the electronics and 3D printing filament, were already owned before the project, so no parts were purchased. The entire assembled robot is shown below:



3 Control Algorithm

The design of the robot is shown below. The robot has one servo motor on each side, with the speeds V_L and V_R, as well as a distance sensor on the right that measures D_R, and a distance sensor on the front that measures D_F.



The velocity of each wheel is determined by the following steering mixing equations:

$$V_L = \text{base speed} + \text{steering value}$$

$$V_R = \text{base speed} - \text{steering value}$$

Therefore, a positive steering value will cause the robot to turn left, and a negative steering value will cause the robot to turn right. A zero steering value will cause the robot to drive in a straight line at the base speed.

The control algorithm is based on a wall follower, which aims to keep D_R constant while driving forwards. This is done by means of a PD (proportional-derivative) controller. This controller subtracts a setpoint value from the value measured by the right-hand sensor, and employs a proportional and derivative term to convert the error to the necessary steering value. However, if only the wall following algorithm was used, the robot would run into a wall that was in front of it. To prevent this, the front sensor was added. If D_F is below a certain threshold value, the robot turns left at the maximum steering value. With this control system, the robot can navigate both left and right turns in the maze.

4 Conclusion

Overall, this project was successful. However, there are a few improvements that could be made if this project was done again (and more time was available to work on it). The wall following algorithm worked well in theory, and could account for changes in the shape and size of the maze. However, the controller was not tuned very well, which resulted in steering oscillations. Better tuning could improve damping and reduce these oscillations.

Additionally, because the sensors do not have a linear relationship between distance and voltage, if they are very close to the wall, they will output a voltage that is the same as if they were much farther from the wall. Because of this, the robot would occasionally get stuck driving very close to the wall instead of at the correct setpoint. This could be fixed with changes to the control algorithm, or by moving the right-hand sensor further towards the center of the robot.

Finally, for improved reliability, it would have been good to solder the circuit instead of leaving it on a breadboard.

5 Appendix: Controller Code

```
/*
 * Cooper McAllister
 * April 30, 2026
 * Microcontrollers Project
 *
 */

/* PINS */
// Left Servo: PTE23 - TPM2_CH1
// Right Servo: PTE22 - TPM2_CH0
// Front Sensor: PTE29 - ADC0_SE4b
// Right Sensor: PTE30 - ADC0_SE23


#include <stdio.h>
#include "board.h"
#include "peripherals.h"
#include "pin_mux.h"
#include "clock_config.h"
#include "fsl_debug_console.h"
#include "fsl_slcd.h"

#define SETPOINT 1000 // The sensor value at which the robot should follow the wall
#define FRONT_THRESH 900 // If this value is seen by the front sensor, turn left

void Print2LCD(char *string);

uint8_t newData = 0;
uint16_t adcReadingF;
uint16_t adcReadingR;
uint16_t adcMillivoltsF = 0;
uint16_t adcMillivoltsR = 0;
uint8_t sensorSelect = 0;
uint8_t LCDSelect = 0;

float error;
float last_error;
float direction = 0; // From -50..50; Left is Positive
float baseSpeed = 40; // From 0..100


void ADC0_IRQHandler() {
        // Every time this handler runs, it will alternate between measuring
        // one sensor and the other
        if (sensorSelect) {
                adcReadingR = ADC0->R[0]; // clear the interrupt flag
                // enable ADC interrupt and request conversion on channel 4
                ADC0->SC1[0] = ADC_SC1_AIEN_MASK | ADC_SC1_ADCH(0b100);
        } else {
                adcReadingF = ADC0->R[0]; // clear the interrupt flag
                // enable ADC interrupt and request conversion on channel 23
                ADC0->SC1[0] = ADC_SC1_AIEN_MASK | ADC_SC1_ADCH(0b10111);
        }
        // Set this flag so the main loop knows there's new data to read
        newData = 1;
        // Alternate between sensors
        sensorSelect = !sensorSelect;
}

void PORTC_PORTD_IRQHandler() {
        // This handler runs when the button on the MCU is pressed.
        // It switches which sensor value is displayed on the LCD screen.
        if (PORTC->PCR[3] & PORT_PCR_ISF_MASK) {
                PORTC->PCR[3] |= PORT_PCR_ISF_MASK; // clear the interrupt flag
                LCDSelect = !LCDSelect;
        }
}

void steering(float dir) {
        // Constrain the direction value to the range -50..50
        if (dir > 50) {
```

```
                dir = 50;
        } else if (dir < -50) {
                dir = -50;
        }
        // Steering mixing
        float leftSpeed = baseSpeed - 2*dir;
        float rightSpeed = baseSpeed + 2*dir;
        // Map the speed to the CnV value necessary to give the
        // correct pulse width signal to the servos
        TPM2->CONTROLS[0].CnV = (int)(1496 - rightSpeed/100*(1496-1424));
        TPM2->CONTROLS[1].CnV = (int)(leftSpeed/100*(1594-1522) + 1522);
}

int main(void) {

    /* Init board hardware. */
    BOARD_InitBootPins();
    BOARD_InitBootClocks();
    BOARD_InitBootPeripherals();
#ifndef BOARD_INIT_DEBUG_CONSOLE_PERIPHERAL
    /* Init FSL debug console. */
    BOARD_InitDebugConsole();
#endif

    // Turn on ADC0, TPM2, PORTE, and PORTC
    SIM->SCGC6 |= SIM_SCGC6_ADC0_MASK | SIM_SCGC6_TPM2_MASK;
    SIM->SCGC5 |= SIM_SCGC5_PORTE_MASK | SIM_SCGC5_PORTC_MASK;


    /*** ADC SETUP ***/
    // Connect PTE29 and PTE30 to ADC
    PORTE->PCR[29] = PORT_PCR_MUX(0);
    PORTE->PCR[30] = PORT_PCR_MUX(0);

    ADC0->SC2 = 1; // select correct voltage reference
    ADC0->SC3 = ADC_SC3_AVGE(1) | ADC_SC3_AVGS(1); // enable averaging
    ADC0->CFG1 = ADC_CFG1_MODE(3) | ADC_CFG1_ADIV(2); // 16-bit conversions, BUS CLK/4
    ADC0->CFG2 |= ADC_CFG2_MUXSEL(1); // Select ADxxb channels
    // enable ADC interrupt and request conversion on channel 4
    ADC0->SC1[0] = ADC_SC1_AIEN_MASK | ADC_SC1_ADCH(0b100);
    NVIC_EnableIRQ(ADC0_IRQn);

    /*** BUTTON SETUP ***/
    // Setup PC3 (SW3): Connect to GPIO, pull up, IRQ on falling edge
    PORTC->PCR[3] = PORT_PCR_MUX(1) | PORT_PCR_PE(1) | PORT_PCR_PS(1) | PORT_PCR_IRQC(0b1010);
    NVIC_EnableIRQ(PORTC_PORTD_IRQn);

    /* TPM SETUP */
    MCG->C1 |= MCG_C1_IRCLKEN_MASK;
    MCG->C2 |= MCG_C2_IRCS(1); // select 8 MHz (instead of 2 MHz) for MCGIRCLK
    SIM->SOPT2 &= ~SIM_SOPT2_TPMSRC_MASK;
    SIM->SOPT2 |= SIM_SOPT2_TPMSRC(0b11); // MCGIRCLK selected for TPM0, TPM1, and TPM2
    // Edge-aligned PWM, high-true pulses
    TPM2->CONTROLS[0].CnSC = TPM_CnSC_MSB(1)| TPM_CnSC_MSA(0)
                             | TPM_CnSC_ELSB(1)| TPM_CnSC_ELSA(0);
    TPM2->CONTROLS[1].CnSC = TPM2->CONTROLS[0].CnSC; // set CH1 the same as CH2
    TPM2->SC = TPM_SC_CMOD(1) | TPM_SC_PS(0b011); // enable clock, prescale divide by 8 for 1MHz clock
    TPM2->MOD = 18182; // set MOD for a frequency of about 55Hz

    // Connect PTE22 and PTE23 to TPM2
    PORTE->PCR[22] = PORT_PCR_MUX(3);
    PORTE->PCR[23] = PORT_PCR_MUX(3);
    // Set PTE22 and PTE23 as outputs
    GPIOE->PDDR |= (1 << 22) | (1 << 23);

    // Set the vehicle to drive straight at first
    steering(50);

    while(1) {

            /*** GET DATA ***/

            if(newData) {
                    // Convert the ADC readings to mV
```

```
                // ADC readings are 16 bits. 2^16 = 0x10000
                // The reference voltage was measured as 3.116V in a previous lab
                // (The exact measured values don't matter for the controller as long
                // as they are consistent)
                adcMillivoltsF = adcReadingF*3116/0x10000;
                adcMillivoltsR = adcReadingR*3116/0x10000;
                // Display either sensor value on the screen, selectable with the button
                char txt[5];
                snprintf(txt, 5, "%4d", (LCDSelect) ? adcMillivoltsF : adcMillivoltsR);
                Print2LCD(txt);
                // Clear the newData flag
                newData = 0;
        }

        /*** CONTROLLER ***/

        // Find the error between the measured value and the setpoint
        error = adcMillivoltsR - (float)SETPOINT;
        // This is a simple PD controller that has as an input the error
        // and as an output the steering direction
        direction = error/20.0 + (error - last_error)/20.0;
        // If the front sensor value is high, there is a wall in front of the
        // robot, so it needs to turn fully to the left
        if(adcMillivoltsF > FRONT_THRESH) {
                direction = 50;
        }
        // If the right sensor value is very low, just steer all the way to the
        // right, since the wall is far away.
        // The purpose of this block of code was to reduce noise for low sensor
        // value, but it might not actually have much of an effect.
        if(adcMillivoltsR < 300) {
                direction = -50;
        }

        // Drive the vehicle in the direction specified by the controller
        steering(direction);

        // Set last error to the current error for the D term of the controller
        last_error = error;

    }
    return 0;
}
```